BASE PAPER

AI-Powered API Gateway Intrusion Detection System Using Machine Learning and Behavioral Analysis

Cyber Security / Machine Learning

Abstract

Modern web applications increasingly depend on Application Programming Interfaces (APIs) for communication between clients and backend services. This dependence creates an important attack surface for threats such as SQL Injection, Cross-Site Scripting (XSS), Path Traversal, authentication abuse and excessive request activity. This mini project proposes an API Gateway Intrusion Detection System (IDS) that combines Machine Learning, HTTP security feature extraction, behavioral analysis and rate limiting to detect and control suspicious API traffic in near real time. The system is implemented using FastAPI as the API gateway and a Random Forest classifier from scikit-learn for machine-learning-based attack detection. A set of 23 security-oriented HTTP features is extracted from requests, including payload characteristics, special-character density, payload entropy, SQL-injection indicators, XSS indicators, URL-encoding anomalies, path-traversal indicators and request-frequency behavior. ML prediction and behavioral analysis are combined to produce a risk score and classify requests as Allow, Warn or Block. A thread-safe sliding-window rate limiter can restrict excessive activity, while security events are stored in SQLite. A lightweight security dashboard provides visibility into requests, alerts, blocked traffic and security statistics. An automated test suite is included for representative attack scenarios. The project is intended as an academic prototype for intelligent API security monitoring; final accuracy, precision, recall and F1-score values should be reported only after running the implemented evaluation.

Keywords: API Security; Intrusion Detection System; Machine Learning; Random Forest; Behavioral Analysis; SQL Injection; XSS; Path Traversal; Rate Limiting; FastAPI

1. Introduction

Application Programming Interfaces have become a central communication layer for web applications, mobile applications, microservices and cloud services. APIs often expose business functions and data, making them an attractive target for attackers. Security controls at the API gateway can inspect requests before they reach backend services and can provide a central location for monitoring and enforcement. OWASP identifies API-specific risks such as broken authentication, broken authorization and unrestricted resource consumption.

Traditional signature and rule-based controls are useful for known patterns but may be bypassed by modified or obfuscated inputs. Machine learning can complement deterministic rules by learning relationships among multiple request features, while behavioral analysis can identify unusual activity over time. The proposed project combines these approaches in a single student-friendly prototype.

2. Problem Statement

API gateways receive a continuous stream of HTTP requests containing paths, query parameters, headers and payloads. Manually examining this traffic is impractical, and a small collection of keyword rules may generate false positives or fail to recognize modified attacks. The problem is to design a lightweight gateway security system that can automatically inspect requests, identify suspicious patterns, analyze client behavior, enforce rate limits, store security events and present actionable alerts.

3. Objectives

• Build a FastAPI-based API gateway security layer. • Extract 23 security-oriented features from incoming HTTP requests. • Use a Random Forest classifier to detect malicious and benign requests. • Add client-level behavioral analysis to detect abnormal activity. • Detect representative SQL injection, XSS and path traversal patterns. • Apply a sliding-window rate limiter for excessive request activity. • Store requests, alerts and risk information in SQLite. • Provide a dashboard for security monitoring. • Evaluate the system using accuracy, precision, recall, F1-score and detection latency.

4. Literature / Existing System

Existing API security solutions commonly use web application firewalls, signatures, schema validation, authentication controls and rate limiting. These controls are valuable but can require continuous rule maintenance. Research in

intrusion detection has also explored supervised classification and anomaly detection. The proposed mini project follows a hybrid approach: deterministic indicators provide fast detection for recognizable patterns, a Random Forest model provides learned classification, and behavioral analysis captures request-sequence information.

5. Proposed System

The proposed system places an intelligent IDS at the API gateway. Each incoming request is inspected before being forwarded to the protected API. The gateway extracts security features, obtains an ML prediction, calculates a behavioral risk score, checks the rate limiter and combines these signals into a final decision. Each security event is logged to SQLite and displayed on the monitoring dashboard.

The three main detection layers are: (1) machine learning, (2) behavioral analysis, and (3) deterministic security and rate-limit controls.

6. System Architecture

Client → FastAPI API Gateway → HTTP Feature Extraction → Random Forest + Behavioral Analysis + Security Rules/Rate Limiter → Risk Score → Allow / Warn / Block → Protected API

Parallel monitoring path: Request + Prediction + Risk Score + Alert → SQLite Database → Security Operations Dashboard.

7. Security Feature Extraction

The prototype uses 23 security-oriented features. A suitable feature set is: HTTP method; URL length; path length; query-string length; body length; special-character count; special-character density; payload entropy; digit count; alphabetic-character count; encoded-character count; double-encoding indicator; SQL keyword indicator; SQL operator/pattern indicator; XSS tag/keyword indicator; script/event-handler indicator; path-traversal indicator; null/control-character indicator; header anomaly indicator; request frequency; unique endpoint count; recent suspicious-request count; and client behavioral-deviation score.

The same feature-extraction logic must be used during model training and live inference to prevent feature mismatch.

8. Machine Learning Methodology

A labeled dataset of benign and malicious API/HTTP requests is prepared. The data is cleaned and transformed into the selected features. The dataset is divided into training and testing sets, and a Random Forest classifier is trained. Random Forest is appropriate for an academic prototype because it can model nonlinear relationships, handle many engineered features and provide feature-importance information.

Pipeline: Dataset → Cleaning → Feature Extraction → Encoding/Preprocessing → Train/Test Split → Random Forest Training → Evaluation → Saved Model → Gateway Inference.

9. Behavioral Analysis

Behavioral analysis maintains a short history for each client identifier, such as an IP address or privacy-preserving client key. The system can track request frequency, endpoint diversity, repeated failures and recent suspicious events. A request that appears ambiguous individually can become high risk when it forms part of an abnormal sequence. Time windows should be used so old activity does not permanently affect the baseline.

10. Rate Limiting

A thread-safe sliding-window rate limiter is used to reduce excessive request activity. For demonstration, the gateway may be configured for a maximum of 50 requests in a rolling 60-second window per client. When the limit is exceeded, the gateway returns HTTP 429 (Too Many Requests) and records the event. The threshold is configurable and should be tuned using normal-traffic measurements.

11. Risk Scoring and Decision Logic

The final decision can combine ML evidence, behavioral evidence and deterministic/rate-limit evidence. A simple formulation is: Final Risk = w1 × ML Risk + w2 × Behavioral Risk + w3 × Rule/Rate Risk.

Low risk → ALLOW; medium risk → WARN and log; high risk → BLOCK. The system should record the decision reason, triggered indicators, model output, client identifier and timestamp so that alerts are explainable and auditable.

12. Database and Dashboard

SQLite stores request metadata, classification results, risk scores, alert types, client identifiers, timestamps and actions. A lightweight dashboard can show total requests, allowed requests, warnings, blocked requests, attack categories, suspicious clients and recent alerts. This provides a simple SOC-style interface suitable for a mini-project demonstration.

13. Implementation Environment

Hardware: student laptop/desktop with at least 4 GB RAM recommended. Software: Python 3.x, Visual Studio Code, FastAPI, Uvicorn, scikit-learn, pandas, NumPy, SQLite and HTML/CSS/JavaScript for the dashboard.

Suggested project structure: app.py; detector.py; features.py; behavioral.py; rate_limiter.py; database.py; model.pkl; train_model.py; test_security.py; templates/; static/; logs/; reports/.

14. Experimental Evaluation

Evaluation should use controlled benign and malicious requests. Suggested test categories are benign traffic, SQL injection, XSS, path traversal, authentication-abuse scenarios and excessive-request bursts. Recommended metrics are accuracy, precision, recall, F1-score, false-positive rate and detection latency. A confusion matrix can also be reported.

Important: numerical performance claims must be replaced with values measured from the student's final implementation and test dataset. Do not claim 98% accuracy, 97% precision, 99% recall or 98% F1-score unless those values are actually obtained during evaluation.

15. Expected Results

The expected result is a working prototype that can inspect API traffic, classify suspicious requests, detect abnormal client behavior, enforce rate limits, record security events and display alerts. The demonstration should include a normal request that is allowed, a malicious request that is detected and blocked, and a request burst that triggers HTTP 429 rate limiting.

16. Advantages

• Centralized API traffic inspection. • Combines ML, behavioral analysis and deterministic controls. • Supports multiple representative attack classes. • Produces risk scores and alert reasons. • Provides historical security records. • Includes a dashboard for easy demonstration. • Lightweight enough for a student laptop. • Extensible to other ML models and security controls.

17. Limitations

The model depends on the quality and representativeness of the training data. Performance may degrade when real traffic differs from training data. Behavioral baselines may produce false positives during legitimate traffic spikes. The project is an academic prototype and should not be represented as a production-grade security gateway without additional security testing, scalability testing, authentication/authorization integration, secure deployment and adversarial ML evaluation.

18. Future Enhancement

Future work can include anomaly-detection and deep-learning models, Redis-based distributed rate limiting, JWT/OAuth-aware controls, API schema validation, explainable AI, Docker deployment, centralized SIEM integration, threat-intelligence feeds and automated incident response.

19. Conclusion

This project proposes an AI-powered API Gateway Intrusion Detection System that combines machine learning, behavioral analysis and rate limiting. The FastAPI-based gateway provides a practical location to inspect requests before they reach backend services, detect suspicious activity, control excessive traffic and expose security events through a dashboard. The hybrid design provides a strong foundation for a final-year mini project and can be evaluated systematically using standard classification and performance metrics.

20. Technology Stack

Component	Technology
API Gateway	FastAPI + Python

ML	Random Forest / scikit-learn
Database	SQLite
Dashboard	HTML + CSS + JavaScript / Python
Testing	Automated API security tests
Development	Visual Studio Code

21. References

[1] OWASP Foundation, OWASP API Security Top 10 – 2023, OWASP API Security Project.

[2] Scikit-learn Documentation, RandomForestClassifier and model evaluation documentation.

[3] FastAPI Documentation, FastAPI framework reference.

[4] N. Provos et al., literature and industry practice on intrusion detection and web security monitoring.

[5] Python Documentation, standard library and HTTP/web development references.

Note: This is an original academic base-paper draft. Experimental results must be replaced with results from the final implementation.